ARCHITECTURE INFRASTRUCTURE DIRECTIVE · VERSION 1.0

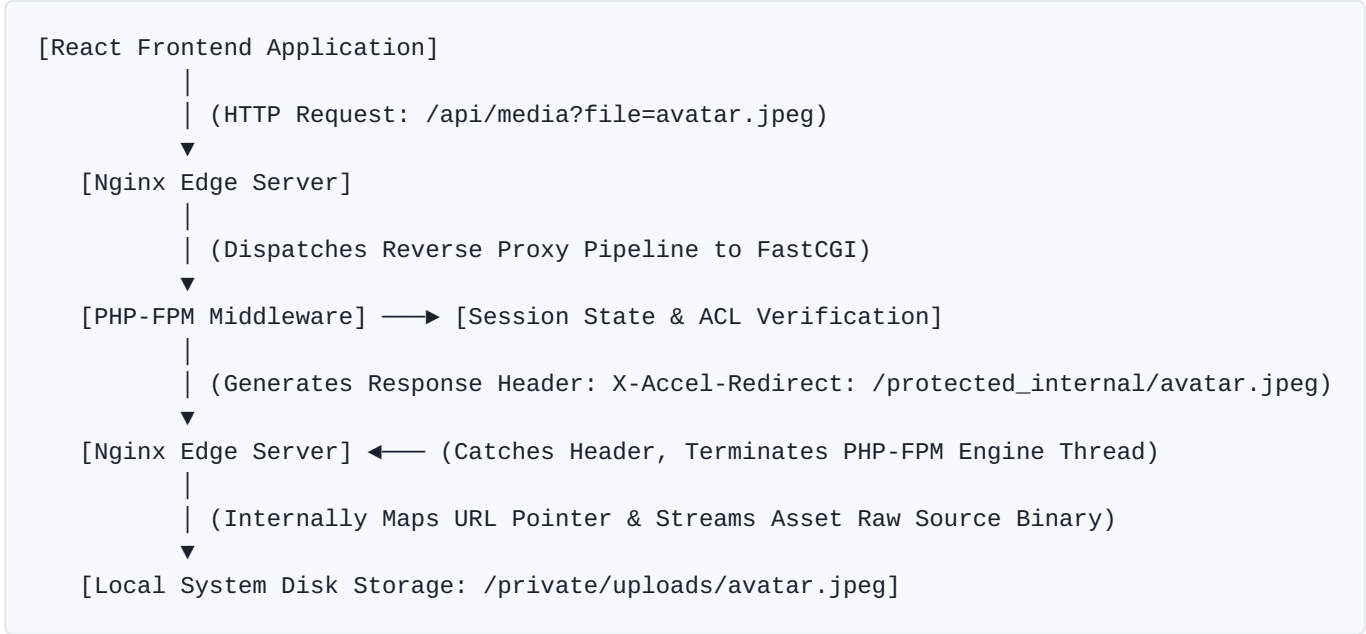
Secure File Delivery Blueprint: Nginx Internal Redirects (X-Accel-Redirect)

1. Executive Summary & Core Objective

To adhere to standard enterprise compliance and zero-trust data segregation policies, all user-uploaded media assets (such as employee avatars, sensitive onboarding documents, and payroll attachments) must be housed entirely outside of the public document web root (`public_html`). The targeted storage location is the local system file directory `/private/uploads/` .

Because the local Nginx instance is configured with persistent extension-level proxy filters, direct relative URL resolution of static image assets located outside the web root triggers native security barriers, resulting in `403 Forbidden` response states. Conversely, traditional software-level routing filters—such as bootstrapping the core PHP framework to continuously stream binary payloads via `readfile()` or `file_get_contents()` — are strictly prohibited due to severe baseline execution penalties, structural CPU bottlenecking, and high memory multi-tenancy overhead.

The Enterprise Solution: Implement a high-performance hybrid delivery system leveraging **Nginx Internal Redirects (`X-Accel-Redirect`)**. This layout decouples runtime security constraints from storage infrastructure: the application layer (PHP) retains absolute administrative oversight for session validation and permission scoping, while offloading actual file IO block manipulation, streaming routines, and high-velocity network packet transfers to Nginx's hyper-optimized C-native static transfer layers.



2. Edge Infrastructure Layer (Nginx Configuration)

You must establish a dedicated, completely isolated, and internal-only location perimeter within the virtual host configuration wrapper. This block explicitly defines the system physical storage layer alias for the requested internal namespace tracking strings while definitively rejecting external browser requests hitting that target namespace URI.

Virtual Host Block Modification

Inject the following routing configuration parameter matrix directly into your active server container scope:

```
# Secure isolation boundary matching the system storage layer pointer
location /protected_internal/ {
    internal;                   # CRITICAL: Rejects public internet entry points (retur
    alias /private/uploads/;    # Direct structural pointer to the secured physical sto

    # Static performance headers for edge-layer optimization
    expires 30d;
    add_header Cache-Control "private, no-transform, must-revalidate";
    add_header X-Content-Type-Options "nosniff";
}
```

> **Mandatory Deployment Protocol:** After updating the virtual host configuration schema, always evaluate infrastructure integrity states by executing `sudo nginx -t`. Once structural validity is formally verified, gracefully cycle the service state via `sudo systemctl reload nginx` to eliminate user-facing platform downtime.

3. Software Control Layer (PHP Authentication Engine)

The backend core environment handles request parsing, processes directory traversal patterns to block path injection vulnerability exploits, verifies resource ownership states via session access control layers, and communicates internal mapping targets to Nginx through clean header injection wrappers.

Asset Scoping Method: UploadHelper.php Modification

Integrate or expose the following logical routine inside your core data transport utility modules:

```php
<?php
/**
 * Processes secure delivery matrices for protected assets via Nginx stream offloading.
 *
 * @param string $filename The target filename identifier requested by the client instar
 */
function serveSecureMedia($filename) {
    // 1. Enforce active authentication/ACL session boundaries
    // Replace with the platform's native enterprise session verification wrapper
    if (!isset($_SESSION['user_id']) || empty($_SESSION['user_id'])) {
        header("HTTP/1.1 403 Forbidden");
        header("Content-Type: text/plain");
        echo "Security Exception: Access Request Denied.";
        exit;
    }

    // 2. Strict cryptographic/string sanitation to suppress directory traversal exploit
    $filename = basename($filename);
    $absolutePath = '/private/uploads/' . $filename;

    // 3. Confirm target resource existence inside physical disk arrays
    if (empty($filename) || !file_exists($absolutePath)) {
        header("HTTP/1.1 404 Not Found");
        header("Content-Type: text/plain");
        echo "Error: Requested asset context could not be located.";
        exit;
    }

    // 4. Resolve Content-Type structures dynamically to avoid cross-site scripting risk
    $mimeType = mime_content_type($absolutePath) ?: 'application/octet-stream';
    header("Content-Type: " . $mimeType);

    // 5. Construct structural internal redirect pointer mapping to the internal locatio
    $internalUri = '/protected_internal/' . $filename;

    // 6. Inject the execution offloader header and forcefully terminate the PHP lifecyc
    header("X-Accel-Redirect: " . $internalUri);
    exit;
}
```

4. Interface Translation Layer (React Frontend)

To preserve visual layout stability and prevent breaking user interface rendering trees, the React frontend application must adapt from using hard-coded direct folder relative paths to using highly maintainable, unified parameter-driven URI builders.

Step 1: Global Media Resolution Assembly

Declare a global helper asset wrapper within your shared utilities toolkit directory (e.g., `src/utils/mediaHelper.js`):

```
/**
 * Converts a raw system asset name into an authenticated API endpoint pointer.
 * @param {string} filename - The asset identifier stored in database records.
 * @returns {string} The relative resource tracking path pointing to the API handler.
 */
export const getSecureMediaUrl = (filename) => {
  if (!filename) return '/assets/placeholders/avatar-default.png';
  return `/api/media?file=${encodeURIComponent(filename)}`;
};
```

Step 2: Core View Rendering Component Architecture

Refactor user view components to cleanly ingest the abstract path generation helper:

```
import React from 'react';
import { getSecureMediaUrl } from '../utils/mediaHelper';

const EmployeeAvatar = ({ employee }) => {
  return (
    <div className="avatar-wrapper">
      <img
        src={getSecureMediaUrl(employee.avatar_file)}
        alt={`${employee.fullName}'s profile`}
        className="ui-avatar-img"
        loading="lazy"
        onError={(e) => {
          // Gracefully handle missing files or authentication validation failure states
          e.target.src = '/assets/placeholders/avatar-default.png';
        }}
      />
    </div>
  );
};

export default EmployeeAvatar;
```

5. Integration Quality Verification & Testing Matrix

Execute the following regression test loops post-deployment to ensure system stability, complete logic validation, and tight infrastructure alignment:

STATUS	TARGET EXECUTION PARAMETER MATRIX	EXPECTED SYSTEM BEHAVIORAL STATE OUTPUT
[ ]	Direct Public Namespace URL Attack Loop `https://domain.com/protected_internal/avatar.jpeg`	Nginx edge layers must intercept the endpoint directly, dropping the evaluation request cycle and immediately throwing a **404 Not Found** or **403 Forbidden** response to the agent browser.
[ ]	Unauthenticated Guest Access Profile Test `https://domain.com/api/media?file=avatar.jpeg`	The PHP session tracker must identify the unauthenticated state, halt framework processing sequences, and return an explicit **403 Forbidden** response state to the client interface.
[ ]	Authorized Multi-Role Operational Flow Verification	The logged-in user interface renders media files seamlessly. Under network analytics tabs, data elements flow with zero frame stalling, standard browser caching layers are active, and memory utilization on the host server remains flat.
[ ]	Directory Traversal & Arbitrary Injection Injection Exploits `/api/media?file=../../../../etc/passwd`	The application layer sanitizes the path structure down to a safe basename variant, fails to match a valid asset within the isolated folder boundary, and cleanly returns a **404 Not Found** sequence.